

İadeAjan — Proje Spesifikasyonu

KOBİ'lerin devlette bekleyen KDV iadelerini analiz eden, iade uygunluk skoru veren ve partner faktoring üzerinden ön finansman teklifi sunan çok ajanlı yapay zeka sistemi.

Proje Özeti

Alan	Detay
Ürün Adı	İadeAjan
Hedef Kitle	Türkiye'deki KDV mükellefi KOBİ'ler
Yarışma	BTK Akademi Hackathon 2026 (Finans teması)
Teknoloji Stack	Python, LangGraph, LangChain, FastAPI, Streamlit
Veri Kaynakları	Paraşüt API, GiB e-Arşiv (mock + gerçek)
Mimari	Multi-Agent Supervisor + 3 Worker Agent

Problem Tanımı

Neden Bu Problem?

Türkiye'de KDV iadesi süreci KOBİ'ler için 3 büyük acı barındırıyor:

- Süre Sorunu:** Ortalama KDV iadesi 40–45 gün sürüyor. Nakit ihtiyacı olan KOBİ bu süre boyunca finansman baskısı altında kalıyor.
- Mevzuat Karmaşıklığı:**
 - 2025 Mart'ta asgari iade talep tutarı 2.000 TL → 10.000 TL'ye yükseltildi
 - 2026'da indirimli oran KDV iade alt limiti 164.000 TL
 - 50.000 TL'nin altındaki iadeler teminatsız alınabiliyor (YMM raporu gerekmez)
 - Tevkifat ve tam istisna iadeleri: işlem yılını izleyen 2. takvim yılı sonuna kadar talep edilmezse hak KAYBI
- Hak Kaybı Riski:** Pek çok KOBİ, süreçleri takip etmek için YMM/muhasebeciye para ödüyor ya da zamanaşımına düşerek hakkını kaybediyor.

Ne Yapmıyor Rakipler?

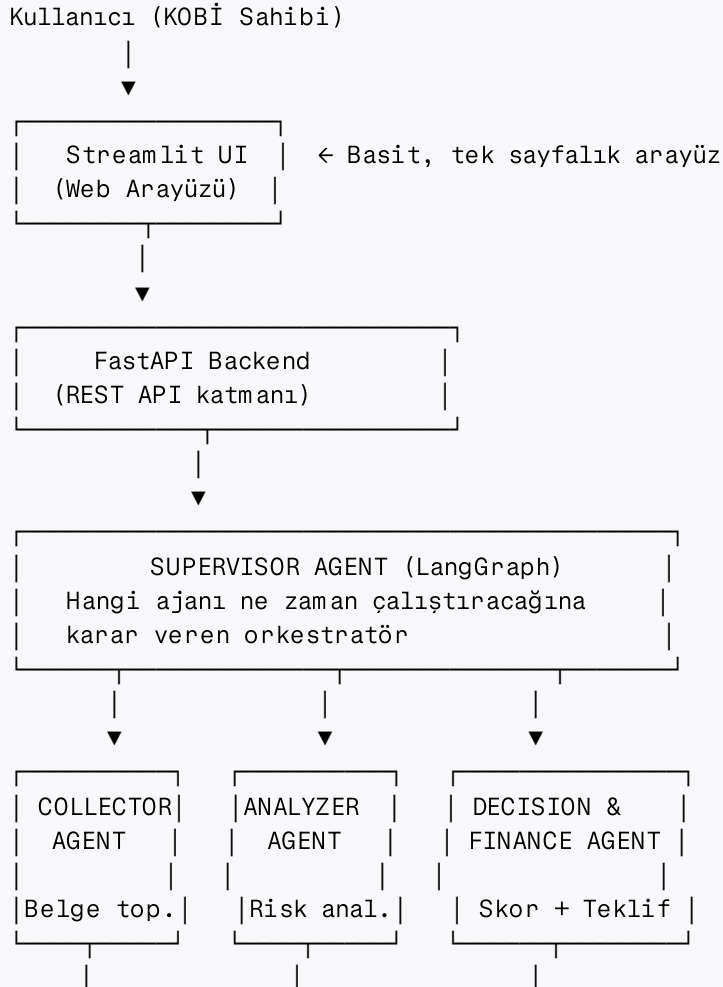
Rakip	Ne Yapıyor	Eksik Ne?
KPMG + Alphabots	Büyük firmalar için belge işleme	KOBİ self-service yok, skor yok, faktoring yok
Paraşüt / KolayBi	Muhasebe ve e-fatura yönetimi	KDV iade analizi yok
TEB Faktoring	Fatura bazlı faktoring	KDV iade hakkı değerlendirmesi yok
KPMG VAT Refunder	Liste hazırlama, hata tespiti	"İade alınır mı?" kararı vermiyor

İadeAjan'ın farkı: Bu dört özelliği ilk kez aynı üründe birleştiriyor:

- ✓ KOBİ self-service
- ✓ KDV uygunluk skoru (0-100)
- ✓ Açıklamalı risk raporu
- ✓ Partner faktoring ön onayı

☒ Sistem Mimarisi

Genel Akış



▼
Paraşüt API
GİB e-Arşiv

▼
Mevzuat RAG
(Vektör DB)

▼
Faktoring
Ön Onay

Multi-Agent Mimari Seçimi: Neden LangGraph?

LangGraph, Supervisor + Worker mimarisini destekliyor:

- Supervisor agent, state machine gibi davranıyor
- Worker agentlar paralel veya sıralı çalışabiliyor
- Her adım loglanıyor → demo'da "hangiajan ne yaptı" gösterilebilir
- BTK Hackathon 2025'te ikinci olan STOX projesi de LangGraph kullandı

☒ Ajan Detayları

Ajan 1: Collector Agent (Belge Toplayıcı)

Görevi: KOBİ'nin vergi ve fatura verilerini ilgili kaynaklardan çekmek.

Yaptığı İşler:

1. Paraşüt API bağlantısı kur (client_id + client_secret)
2. Belirli tarih aralığındaki e-faturaları çek (hem gelen hem giden)
3. İade türünü tespit et:
 - İhracattan doğan KDV iadesi mi?
 - Tevkifattan doğan KDV iadesi mi?
 - İndirimli orandan doğan KDV iadesi mi?
4. Gerekli belge listesini oluştur (iade türüne göre)
5. Toplanan veriyi Analyzer Agent'a ilet

Çekilen Veriler:

- Satış faturaları (giden e-faturalar)
- Alış faturaları (gelen e-faturalar)
- Tedarikçi listesi
- KDV beyanname dönemleri
- İhracat işlemi var mı? (mock veya API)

Teknik:

```
# Paraşüt API entegrasyonu
class CollectorAgent:
    def __init__(self, parasut_client_id, parasut_client_secret):
        self.base_url = "https://api.parasut.com/v4"
        self.headers = self._get_auth_headers()

    def fetch_invoices(self, company_id, start_date, end_date):
```

```
# GET /v4/{company_id}/sales_invoices
# GET /v4/{company_id}/purchase_bills
pass

def detect_refund_type(self, invoices):
    # İhracat, tevkifat, indirimli oran tespiti
    pass
```

Mock Fallback: Paraşüt API erişimi yoksa, `mock_data/celik_as_invoices.json` dosyasından demo verisi yüklenir. Demo günü güvenli.

Ajan 2: Analyzer Agent (Risk Analizci)

Görevi: Toplanan belge setini mevzuata karşı kontrol etmek, risk puanları çıkarmak.

Yaptığı İşler:

1. Tedarikçi uyumsuzluklarını kontrol et
 - Tedarikçi, aynı dönemde KDV beyannamesi vermiş mi?
 - Sahte belge riski var mı?
2. Belge eksiksizliğini kontrol et
 - İhracat iadesi için: gümrük çıkış beyannamesi var mı?
 - Tevkifat iadesi için: 2 no'lu KDV beyannamesi var mı?
3. Zamanaşımı kontrolü
 - Hangi dönemler talep edilmemiş?
 - Kaç gün kalmış?
4. Limit kontrolü
 - Tutar 50.000 TL altı mı? → Teminatsız iade (kolay)
 - Tutar 164.000 TL altı mı? → İndirimli oran limiti
 - YMM raporu gerekiyor mu?
5. VDK risk skoru tahmini
 - Bu dosya incelemeye girer mi?

RAG (Retrieval Augmented Generation) Yapısı:

Vektör Veritabanı (ChromaDB / FAISS):

- KDV Genel Uygulama Tebliği (güncel)
- 2026 KDV iade limitleri tablosu
- 50.000 TL teminatsız iade kuralı
- YMM tam tasdik limitleri
- Zamanaşımı süreleri (2+5 yıl kuralı)
- VDK risk analiz kriterleri

Çıktısı:

```
{
  "risk_items": [
    {
      "type": "tedarikci_uyumsuzlugu",
      "severity": "yüksek",
      "description": "Tedarikçi X, 2024/Q3 döneminde KDV beyannamesi vermemiş",
    }
  ]
}
```

```
{
  "affected_invoices": ["F-2024-0342", "F-2024-0351"],
  "fix_suggestion": "Bu faturalar için alternatif tedarikçi belgesi talep edin",
},
{
  "type": "zamanasimi_uyarisi",
  "severity": "kritik",
  "description": "2024 yılı tevkifat iadesi için son başvuru: 31.12.2026",
  "days_remaining": 230,
  "amount_at_risk": 45000
}
],
"missing_documents": ["Gümrük çıkış beyannamesi", "İhracatçı listesi"],
"base_score": 72
}
```

Ajan 3: Decision & Finance Agent (Karar ve Finansman Ajanı)

Görevi: Risk analizini final skora dönüştürmek ve uygun finansman teklifini üretmek.

Yaptığı İşler:

1. Analyzer'dan gelen risk_items'ı puanla
 - Her yüksek risk: -10 puan
 - Her orta risk: -5 puan
 - Her düşük risk: -2 puan
 - Her eksik belge: -8 puan
2. Final İade Uygunluk Skoru üret (0-100)
3. Skor bandına göre karar ver:
 - 85-100: "Yüksek uygunluk - Finansman ön onayı aktif"
 - 65-84: "Orta uygunluk - Eksikler giderilmeli"
 - 0-64: "Düşük uygunluk - YMM desteği önerilir"
4. Finansman teklifi oluştur (skor $\geq$ 65 ise)
 - İade tutarının %80-90'ı kadar avans
 - Faktoring faiz oranı tahmini
 - Ön onay referans kodu üret
5. Öneri listesi oluştur (her risk item için)

Skor Hesaplama Mantığı:

```
def calculate_score(risk_items, missing_docs, base_score=100):
    deductions = {
        "yüksek": 10,
        "orta": 5,
        "düşük": 2,
        "kritik": 15
    }
    score = base_score
    for item in risk_items:
        score -= deductions.get(item["severity"], 5)
    score -= len(missing_docs) * 8
    return max(0, min(100, score))
```

Çıktısı:

```
{
  "iade_uygunluk_skoru": 88,
  "skor_aciklamasi": "Yüksek uygunluk - 1 orta risk tespit edildi",
  "finansman_teklifi": {
    "iade_tutari_tahmini": 320000,
    "avans_orani": 0.90,
    "avans_tutari": 288000,
    "onay_durumu": "Ön Onay Verildi",
    "referans_kodu": "IA-2026-0042"
  },
  "oneri_listesi": [
    "Tedarikçi X'ten alternatif belge talep edin → Skor +7 puan artacak",
    "Gümrük çıkış beyannamesi ekleyin → Skor +8 puan artacak"
  ],
  "zamanasimi_uyarilari": [
    "2024 tevkifat iadesi için 230 gün kaldı - Acele edin!"
  ]
}
```

☒ Proje Klasör Yapısı

```
iadeajan/
├── README.md                ← Bu dosya
├── requirements.txt          ← Python bağımlılıkları
├── .env.example              ← API key şablonu
├── agents/
│   ├── __init__.py
│   ├── supervisor.py        ← LangGraph supervisor (orkestratör)
│   ├── collector_agent.py   ← Belge toplama ajanı
│   ├── analyzer_agent.py    ← Risk analiz ajanı
│   └── decision_agent.py    ← Skor ve finansman ajanı
├── tools/
│   ├── __init__.py
│   ├── parasut_client.py    ← Paraşüt API wrapper
│   ├── gib_client.py        ← GİB e-Arşiv client (mock destekli)
│   ├── score_calculator.py ← Skor hesaplama motoru
│   └── rag_retriever.py     ← Mevzuat RAG sistemi
├── knowledge_base/
│   ├── kdv_genel_tebliğ.txt ← KDV Genel Uygulama Tebliği metni
│   ├── iade_limitleri_2026.json ← 2026 güncel limitler
│   ├── zamanasimi_kurallari.txt ← Zamanaşımı mevzuatı
│   └── risk_kriterleri.txt   ← VDK risk analiz kriterleri
├── mock_data/
│   ├── celik_as_invoices.json ← Demo şirketi fatura verileri
│   ├── celik_as_suppliers.json ← Tedarikçi listesi
│   └── demo_scenario.json    ← Demo için hazır senaryo
```

```

├── api/
│   ├── __init__.py
│   ├── main.py                ← FastAPI uygulama
│   └── routes/
│       ├── analyze.py        ← POST /analyze endpoint
│       └── health.py         ← GET /health endpoint
├── ui/
│   ├── app.py                ← Streamlit ana uygulama
│   ├── components/
│   │   ├── score_card.py     ← Skor kartı bileşeni
│   │   ├── risk_report.py    ← Risk raporu bileşeni
│   │   └── finance_offer.py  ← Finansman teklifi bileşeni
│   └── assets/
│       └── logo.png
└── tests/
    ├── test_collector.py
    ├── test_analyzer.py
    └── test_decision.py

```

☒ API Entegrasyonları

Paraşüt API

```

# .env dosyası
PARASUT_CLIENT_ID=your_client_id
PARASUT_CLIENT_SECRET=your_client_secret
PARASUT_COMPANY_ID=your_company_id

# Temel endpoint'ler
BASE_URL = "https://api.parasut.com/v4/{company_id}"

# Kullanılacak endpoint'ler:
GET /sales_invoices      # Satış faturaları (gelen KDV)
GET /purchase_bills     # Alış faturaları (ödenen KDV)
GET /contacts           # Tedarikçi/müşteri listesi

```

GİB e-Arşiv (Mock Fallback)

```

# Gerçek ortam: GİB API'sine Paraşüt entegrasyonu üzerinden eriş
# Demo/hackathon: mock_data/ceLIK_as_invoices.json kullan

USE MOCK = os.getenv("USE MOCK", "true") # Hackathon için default true

```

LangGraph Supervisor Kurulumu

```
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode

# State tanımı
class IadeAjanState(TypedDict):
    company_id: str
    date_range: dict
    collected_data: dict      # Collector çıktısı
    risk_analysis: dict       # Analyzer çıktısı
    final_score: int          # Decision çıktısı
    finance_offer: dict       # Finansman teklifi
    messages: list

# Supervisor graph
workflow = StateGraph(IadeAjanState)
workflow.add_node("collector", collector_agent)
workflow.add_node("analyzer", analyzer_agent)
workflow.add_node("decision", decision_agent)

workflow.set_entry_point("collector")
workflow.add_edge("collector", "analyzer")
workflow.add_edge("analyzer", "decision")
workflow.add_edge("decision", END)

app = workflow.compile()
```

☒ Kullanıcı Arayüzü (Streamlit)

Sayfa Akışı

- GİRİŞ SAYFASI
 - └ Şirket adı gir
 - └ Paraşüt API bağlan (veya "Demo Modu" seç)
 - └ Analiz Dönemini Seç (tarih aralığı)
- ANALİZ EKRANI (Loading)
 - └ [■■■■] Faturalar toplanıyor...
 - └ [■■■■] Risk analizi yapılıyor...
 - └ [■■■■] Skor hesaplanıyor...
- SONUÇ EKRANI
 - └ KDV İADE UYGUNLUK SKORU: 88/100 (büyük, renkli)
 - └ Risk Raporu Kartları
 - └ ☒ Kritik Riskler
 - └ ☒ Orta Riskler
 - └ ☒ Düşük Riskler
 - └ Öneri Listesi ("Şunu yap → +X puan")
 - └ Zamanaşımı Uyarıları
 - └ FİNANSMAN TEKLİFİ KARTI
 - └ Tahmini İade: 320.000 ₺

└─ Avans Tutarı: 288.000 ₺ (%90)
└─ [ÖN ONAYI KABUL ET] butonu

Renk Kodlaması

Skor Aralığı	Renk	Mesaj
85-100	☑ Yeşil	Yüksek Uygunluk
65-84	☑ Sarı	Orta Uygunluk – Düzeltilebilir
40-64	☑ Turuncu	Düşük Uygunluk – Destek Alın
0-39	☑ Kırmızı	Yüksek Risk – YMM Gerekli

☑ Demo Senaryosu (BTK Hackathon 7 Dakika)

Sahne Planı

☑ 0:00-1:00 | Problem Anlatımı

"Türkiye'de 370 milyar TL devlette kilitli.
KDV iadesi 40-45 gün sürüyor.
KOBİ'ler ya hak kaybı yaşıyor ya YMM'ye para ödüyor."
→ Slide: Rakamlar + acı nokta

☑ 1:00-1:30 | Çözüm Tanıtımı

"İadeAjan: 3 yapay zeka ajanı, 3 dakikada cevap veriyor."
→ Mimari diyagramı göster (Supervisor + 3 worker)

☑ 1:30-4:00 | CANLI DEMO

- "Çelik A.Ş." demo hesabına gir
- "Analiz Başlat" butonuna bas
- Ekranda ajanlar sırayla çalışıyor (animasyon)
- Skor çıkıyor: 88/100
- Risk raporu: "Tedarikçi uyumsuzluğu düzeltilirse skor 95'e çıkar"
- Finansman kartı: "288.000 ₺ ön avans - Ön Onay Verildi"

☑ 4:00-5:30 | Pazar ve İş Modeli

TAM (Toplam Adreslenebilir Market): Türkiye'deki KDV mükellefi KOBİ'ler
Gelir modeli:

- SaaS: Aylık 299 ₺/şirket
- Başarı komisyonu: İade tutarının %0.75'i
- Faktoring referans fee: Partner gelir paylaşımı

☑ 5:30-7:00 | Takım + Roadmap

MVP → Pilot (50 KOBİ) → Ölçek → YMM entegrasyonu

☒ requirements.txt

```
langchain==0.3.x
langgraph==0.2.x
langchain-openai==0.2.x
langchain-community==0.3.x
chromadb==0.5.x
fastapi==0.115.x
uvicorn==0.32.x
streamlit==1.40.x
httpx==0.28.x
python-dotenv==1.0.x
pydantic==2.10.x
faiss-cpu==1.9.x
openai==1.57.x
pytest==8.3.x
```

☒ .env.example

```
# LLM
OPENAI_API_KEY=sk-...
# Alternatif: Google Gemini (BTK x Google ortaklığı nedeniyle tercih edilebilir)
GOOGLE_API_KEY=...

# Paraşüt API
PARASUT_CLIENT_ID=
PARASUT_CLIENT_SECRET=
PARASUT_COMPANY_ID=

# Demo Modu
USE MOCK=true          # true: mock data kullan, false: gerçek API

# LangSmith (ajan adımlarını logla - demo için çok işe yarar)
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=
LANGCHAIN_PROJECT=iadeajan
```

☒ Kurulum ve Çalıştırma

```
# 1. Repo'yu klon la
git clone https://github.com/kullanici/iadeajan.git
cd iadeajan

# 2. Virtual environment
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# 3. Bağımlılıkları yükle
pip install -r requirements.txt
```

```
# 4. .env dosyasını hazırla
cp .env.example .env
# .env dosyasını düzenle, API key'leri gir

# 5. Knowledge base'i yükle (RAG)
python tools/rag_retriever.py --build

# 6. Backend'i başlat
uvicorn api.main:app --reload --port 8000

# 7. UI'ı başlat (yeni terminal)
streamlit run ui/app.py
```

İş Modeli ve Pazar

Gelir Akışları

Gelir Türü	Açıklama	Tahmini Gelir
SaaS Abonelik	Aylık 299–499 ₺/şirket	Ölçeklenebilir, öngörülebilir
Başarı Komisyonu	İade tutarının %0.75'i	Büyük iadelerden yüksek gelir
Faktoring Referans	Partner gelir paylaşımı	Pasif gelir kaynağı
B2B Lisans	YMM/SMMM firmalara	Kurumsal satış kanalı

Büyüme Yol Haritası

- ☒ Hackathon (Mayıs 2026)
 - └ MVP: Demo senaryosu + çalışan sistem
- ☒ Pilot (Haziran–Ağustos 2026)
 - └ 50 KOBİ ile beta test
 - └ Paraşüt ile resmi entegrasyon
- ☒ Lansman (Eylül 2026)
 - └ Faktoring partneri anlaşması
 - └ YMM entegrasyon modülü
- ☒ Ölçek (2027)
 - └ GİB API resmi entegrasyon
 - └ Muhasebe yazılımları ile white-label

⚠️ Önemli Notlar

Hukuki Sorumluluk Sınırı

İadeAjan bir karar destek sistemidir, resmi vergi danışmanı değildir. Tüm çıktılar:

- "Uygunluk Skoru" olarak sunulur, "garanti" değil
- Nihai kararlar kullanıcının ve YMM'nin sorumluluğundadır
- Sistem "kesin alınır" değil, "yüksek uygunluk" dili kullanır

Demo Güvenliği

- `USE MOCK=true` ile tüm API bağımlılıkları ortadan kalkar
- Demo sırasında internet kesilse bile sistem çalışır
- Mock data `mock_data/ceLIK_as_invoices.json` üzerinden gelir

BTK Hackathon Gereksinimleri

- ✔ Canlı çalışan link zorunlu → Streamlit Cloud veya Railway.app deploy edilecek
- ✔ GitHub repo public olmalı
- ✔ 7 dakikalık online sunum
- ✔ Agentik yapı açıkça gösterilmeli (mimari diyagramı)

Son güncelleme: Mayıs 2026 / BTK Akademi Hackathon 2026 için hazırlandı